

AGENCYOS

AI Agent Architecture

Document 03 of the AgencyOS Master Documentation Set

SALES • PROJECT MANAGEMENT • DESIGN • PROTOTYPE • DEVELOPMENT • QA • FINANCE •
SUPPORT • CUSTOMER SUCCESS • UPSELL • ORCHESTRATION

Purpose: Define the responsibilities, authority, inputs, outputs, handoffs, quality gates, memory, failure handling and collaboration model for AgencyOS AI agents.

1. Architecture Principle

AgencyOS is a multi-agent operating system. Agents do the work; the Orchestrator coordinates the work; policy controls authority; Admin controls exceptions and high-risk decisions.

- Agents are specialized rather than one giant general-purpose agent.
- Agents may collaborate through structured tasks/events rather than uncontrolled free-form conversation.
- Every agent has an explicit role, scope, tools, inputs, outputs, Definition of Done and escalation path.
- No agent may silently perform an action outside its authority.
- AI autonomy must be enforced by backend/application policy, not only by prompt instructions.
- Every important agent action must be attributable and auditable.

2. Human Roles vs AI Roles

- Human: Super Admin / Owner — ultimate business authority.
- Human: Admin — operational approval authority according to configured policy.
- Human: Client — requirements, feedback, approvals, payments and acceptance.
- AI: Sales Agent.
- AI: Project Manager Agent.
- AI: UI Designer Agent.
- AI: Prototype Agent.
- AI: Developer Agents.
- AI: QA Agent.
- AI: Finance Agent.
- AI: Support Agent.
- AI: Customer Success Agent.
- AI: Upsell Agent.
- AI: Orchestrator / Router Agent.

The architecture must support the same functional role being performed by an AI agent today and by a human employee in the future.

3. Agent Hierarchy

Client/External Channels → Orchestrator → Specialized Agents → Tools/Systems →
Evidence/State → Next Agent/Approval Gate

- The Orchestrator should not become the business owner of every task. It coordinates routing, dependencies, retries, context and escalation.
- Specialized agents own domain execution.
- Admin approval is an explicit control layer between AI intent and governed action.
- The database/workflow engine remains the authoritative source of state.

4. Shared Agent Contract

Every agent must define

- Purpose and business responsibility.
- Allowed inputs.
- Expected outputs.

- Allowed tools/actions.
- Forbidden actions.
- Applicable policies.
- Required approvals.
- Definition of Done.
- Failure and retry strategy.
- Escalation conditions.
- Required evidence.
- Memory/context requirements.
- Downstream handoff contract.

Standard execution pattern

RECEIVE TASK → LOAD CONTEXT → CHECK POLICY → PLAN → EXECUTE → VERIFY →
RECORD EVIDENCE → UPDATE STATE → HANDOFF / ESCALATE

An agent must not update a terminal workflow state until its required verification criteria are satisfied.

5. Sales Agent

Mission

Convert qualified leads into clients while collecting accurate requirements and building trust.

Responsibilities

- Respond to new WhatsApp leads.
- Detect and follow the client's language.
- Support text and voice-note conversations.
- Qualify the lead.
- Discover requirements and project intent.
- Identify service/project type.
- Ask for references and examples.
- Identify budget/urgency/constraints.
- Send approved samples, portfolio, previous work and demos.
- Follow up automatically.
- Detect objections and trust concerns.
- Prepare quotation/proposal.
- Handle negotiation within policy.
- Recommend discounts/offers only within approved commercial boundaries.
- Request Admin approval for governed exceptions.
- Mark opportunity WON/LOST according to valid workflow evidence.

Sales Agent must NOT

- Invent prices or commercial commitments outside policy.
- Promise unsupported delivery dates/features.
- Approve its own out-of-policy discount.
- Mark an ambiguous client response as a binding acceptance.

Handoff

WON → Onboarding/Project Manager Agent with structured client, scope, quotation, negotiation and approval context.

6. Project Manager Agent

Mission

Own operational orchestration of each active project after sales conversion.

Responsibilities

- Create/maintain project lifecycle state.
- Coordinate the project WhatsApp group.
- Coordinate agents and handoffs.
- Track requirements, scope, milestones and deadlines.
- Prepare client review messages.
- Track client approvals and feedback.
- Coordinate UI, prototype, development and QA stages.
- Prepare milestone completion requests.
- Trigger invoice workflows when approval conditions are met.
- Request Admin payment verification.
- Announce verified payments and project starts.
- Detect blockers and escalate.
- Maintain the project timeline.

Project Manager must NOT

- Treat unverified payment as paid.
- Override scope or commercial policy silently.
- Declare completion when Definition of Done is not met.
- Skip required client/Admin approvals.

7. UI Designer Agent

Mission

Transform approved requirements into professional, scope-complete UI designs.

Responsibilities

- Read the approved scope and requirement set.
- Identify all relevant screens and flows.
- Generate multiple theme/color directions when appropriate.
- Design all scoped customer/vendor/delivery/admin interfaces.
- Ensure every scoped feature has corresponding UI representation.
- Preserve design versions.
- Prepare client-review package.
- Apply approved revisions.
- Produce design evidence suitable for prototype handoff.

Quality gate

- Scope coverage check.
- Screen/feature completeness check.
- Consistency/UX review.
- Version integrity.
- Admin approval where configured before client release.
- Client approval must reference an exact design version.

8. Prototype Agent

Mission

Convert approved UI into a functional prototype that demonstrates the intended user experience.

- Consume approved UI and scope.
- Create APK/web prototype artifacts appropriate to the project.
- Run smoke checks.
- Version each prototype build.
- Prepare client review package.
- Collect and structure client feedback.
- Fix prototype issues and rebuild.
- Hand approved prototype context to development agents.

Prototype Agent must not silently add production scope that was not approved.

9. Developer Agent System

Mission

Implement the approved product reliably and continuously until the engineering Definition of Done is met.

Specializations

- Frontend/Web Developer Agent.
- Mobile/Android Developer Agent.
- Backend/API Developer Agent.
- Database Developer Agent.
- Integration Developer Agent.
- DevOps/Deployment Agent.
- Security-focused Developer Agent.
- Refactoring/Performance Agent where useful.

Developer workflow

REQUIREMENTS → TASK BREAKDOWN → IMPLEMENT → TEST → BUILD → SELF-REVIEW
→ QA HANDOFF

- Developers may create branches, commits, tests and pull requests.
- Production merge/deployment remains approval controlled.
- Developer agents must report exact changes, tests and known limitations.
- Incomplete implementation must remain incomplete; never claim success to move the workflow forward.

10. QA Agent

Mission

Independently verify that the delivered system actually works and meets defined acceptance criteria.

Responsibilities

- Generate test plans.
- Functional testing.
- UI/UX testing.
- API testing.
- Database testing.
- Authorization/RLS and tenant isolation testing.
- Payment/invoice testing.
- Webhook/retry/idempotency testing.
- Concurrency testing where relevant.
- Regression testing.
- Device/browser testing.
- Performance checks.
- Security checks.
- Build and deployment smoke tests.
- Create and verify bugs.
- Produce production-readiness recommendation.

QA independence

A developer saying 'done' is not QA evidence. QA must independently verify the result.

11. Finance Agent

Mission

Operate invoice/payment workflows safely while preserving financial correctness.

- Generate invoices from approved commercial terms.
- Ask/record GST vs non-GST billing preference.
- Prepare approved payment instructions.
- Track payment status.
- Record payment proofs and references.
- Create Admin verification requests.
- Record verified payment.
- Generate receipts/financial records where applicable.
- Track outstanding payments and reminders.
- Support reconciliation.

Critical constraints

- Never convert database errors into zero/empty payment totals.
- Never allow concurrent operations to violate invoice/payment invariants.
- Use transactions, idempotency and uniqueness where required.
- Never treat a client screenshot/message as final payment verification when Admin verification is required.

12. Support Agent

Mission

Provide routine client support after and during delivery without exposing internal information.

- Understand client issue in the client's language.
- Identify whether issue is support, bug, change request or new project.
- Answer known questions from approved knowledge.
- Create support tickets/tasks.
- Escalate technical issues to Developer/QA agents.
- Escalate commercial issues to Project Manager/Sales/Admin as appropriate.
- Maintain support history.

13. Customer Success Agent

Mission

Protect the client relationship after delivery and identify legitimate retention opportunities.

- Monitor client engagement and satisfaction signals.
- Track maintenance needs.
- Detect dormant/at-risk clients.
- Identify repeat-project signals.
- Prepare customer-success check-ins.
- Coordinate maintenance and support workflows.
- Feed lessons back to the agency analytics system.

Customer Success Agent must not silently create commercial commitments.

14. Upsell Agent

Mission

Identify relevant additional services and create qualified opportunities without becoming spammy or uncontrolled.

- Analyze project type, client needs, usage, support requests and history.
- Identify maintenance, hosting, support, updates, new features, new apps, redesigns, integrations, AI automation or other approved services.
- Prioritize opportunities by relevance and likely value.
- Prepare offers from the approved service catalog.
- Route commercial exceptions to Admin.
- Create a new opportunity when the request is actually a new project.
- Track upsell outcome.

15. Orchestrator / Router Agent

Mission

Coordinate the agent workforce and select the best available AI model/provider for each task within Admin-defined policies.

Responsibilities

- Receive high-level workflow events.

- Select the responsible specialist agent.
- Build structured task packets.
- Provide required context.
- Manage dependencies.
- Route failures.
- Retry safely.
- Select fallback agents/providers.
- Detect deadlocks and blocked tasks.
- Escalate to Admin when policy or authority is unclear.
- Track task provenance.

Model routing

- Sales may use one provider while coding uses another.
- UI generation may use a model optimized for multimodal/design tasks.
- QA may use a model optimized for reasoning/test analysis.
- Finance may use a model with appropriate structured-output reliability.
- Admin configures preferred provider, fallback order, cost/usage limits and availability rules.

Provider selection must never override business authorization.

16. Agent-to-Agent Handoff Protocol

Agents communicate through structured tasks/events rather than relying on conversational memory alone.

Minimum handoff payload

- project_id / client_id / organization_id.
- Source agent and destination agent.
- Task type.
- Current workflow state.
- Approved scope reference.
- Relevant artifact/version IDs.
- Required output.
- Acceptance criteria.
- Deadline/priority if configured.
- Policy decision and approval references.
- Evidence links.
- Known blockers.
- Retry count / previous failure summary.

17. Shared Memory Architecture

- Client memory — stable client profile and communication preferences.
- Lead memory — sales conversation, objections, qualification and history.
- Project memory — requirements, scope, milestones, artifacts, approvals and decisions.
- Agent memory — task-specific context and learned operational patterns where permitted.
- Global business knowledge — approved service catalog, policies, portfolio, FAQs and operating rules.
- Audit memory — immutable/append-only important actions and approvals where architecture permits.

Agents should receive the minimum relevant context needed for their task while preserving continuity.

18. Tool Permissions

Tool access must be role-based and policy-enforced.

- Sales: CRM, approved knowledge, WhatsApp/email communication, quotation drafting, approval requests.
- PM: project management, client communication, milestone workflow, task orchestration, approval requests.
- UI: design/generation tools, artifact storage, project requirements.
- Developer: repository, build/test tools, issue/task system, approved infrastructure tools.
- QA: test environments, logs, test runners, issue system.
- Finance: invoice/payment records, approved account instructions, reconciliation tools.
- Support: ticketing, approved knowledge, client communication.
- Orchestrator: agent routing and task management; not unrestricted business authority.
- No agent receives secrets/tools it does not need.

19. Agent Approval Model

- GREEN — safe routine automation: task creation, internal notifications, reports, reminders, non-destructive calculations.
- YELLOW — routine client operations inside approved policy.
- RED — high-risk actions: out-of-policy pricing/discounts, refunds, cancellation, production deployment, destructive operations, major commercial exceptions, permission/security policy changes and final handover.

The exact boundaries are configurable in the Admin Policy Engine.

20. Agent Failure Handling

RETRY → ALTERNATIVE STRATEGY → SPECIALIST/FALLBACK AGENT →
ORCHESTRATOR ESCALATION → ADMIN

- Retries must be safe and idempotent where side effects exist.
- Do not blindly retry payments, messages, destructive database operations or other non-repeatable actions.
- Record failure reason, provider/model, attempt count and evidence.
- If an agent repeatedly fails, the task must not be marked complete.
- If the failure reveals missing requirements, return to clarification rather than guessing.

21. Agent Quality Gates

- Input validation.
- Policy check.
- Task execution.
- Output validation.
- Schema/structured-output validation.
- Business-rule validation.
- Evidence capture.
- Automated tests where applicable.
- Independent QA for important deliverables.
- Approval gate where required.

- Handoff only after acceptance criteria are satisfied.

22. Agent Observability

- Every execution should record agent, provider/model, task, start/end time, result status, retry count and relevant cost/usage metadata where available.
- Track success/failure rates.
- Track latency.
- Track fallback frequency.
- Track human escalations.
- Track policy rejections.
- Track client-impacting failures.
- Track agent handoff bottlenecks.

Observability is required for improving routing and reliability over time.

23. Agent Security

- Agents operate under least privilege.
- Organization/tenant context must be enforced on every scoped action.
- Secrets must not be placed into prompts unnecessarily.
- Client-facing agents cannot access internal credentials, private strategy or unrestricted administrative data.
- High-risk tool calls require backend authorization and, where configured, Admin approval.
- Agent-generated content must not be treated as trusted input without validation.

24. Agent Completion Contract

An agent may say COMPLETE only when the configured Definition of Done is satisfied and evidence exists.

- Sales: requirements sufficient + quotation state valid + approval/acceptance evidence as required.
- PM: state transition valid + required dependencies satisfied + evidence recorded.
- UI: scope coverage + review + required approval/version.
- Prototype: build generated + smoke test + required review.
- Developer: implementation + tests + build + handoff package.
- QA: required tests passed + critical defects resolved + readiness recommendation.
- Finance: correct invoice/payment state + verification evidence.
- Support: issue resolved or correctly escalated.
- Customer Success/Upsell: qualified opportunity with policy-compliant offer.
- Orchestrator: downstream task accepted or correctly escalated.

25. Recommended Agent Collaboration Example

Sales → PM → UI → Prototype → Developer → QA → Finance/PM milestone gate → Developer → QA → Finance → Handover → Customer Success → Upsell

The actual route may branch based on project type. For example, a food-delivery system may require customer, delivery, vendor and admin interfaces; a simpler website may require only web and admin work.

26. No-Incomplete-Project Rule

The architecture must make it difficult for any single agent to hide incomplete work.

- Developer claims are checked by tests and QA.
- UI claims are checked for scope coverage.
- Prototype claims are checked through actual builds/smoke tests.
- Finance claims are checked through transaction/payment evidence.
- Project Manager claims are checked against workflow state and dependencies.
- Final completion requires all configured gates.
- If a required gate fails, the project remains in the appropriate incomplete state.

27. Final Architecture

CLIENT/WHATSAPP → SALES → ORCHESTRATOR → SPECIALIST AGENTS → TOOLS →
VERIFICATION → WORKFLOW STATE → NEXT AGENT / ADMIN APPROVAL

AgencyOS should behave like an experienced internal agency team: Sales sells, PM coordinates, UI designs, Prototype demonstrates, Developers build, QA verifies, Finance protects the money flow, Support resolves issues, Customer Success retains clients, Upsell expands relationships, and the Orchestrator keeps the whole system moving.

Humans do not micromanage routine work. Humans govern authority, strategy and exceptions.